

SKILL WEEK-6

SESSION-11

Task1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int count = 0;

/* Store a command in history */
void storeCommand(char *command)
{
    if (count < MAX_HISTORY)
    {
        strcpy(history[count], command);
        count++;
    }
    else
    {
        /* Shift old commands to make space */
        for (int i = 1; i < MAX_HISTORY; i++)
        {
            strcpy(history[i - 1], history[i]);
        }
        strcpy(history[MAX_HISTORY - 1], command);
    }
}

/* Display all stored commands */
void displayHistory()
{
    if (count == 0)
    {
        printf("\nHistory is empty.\n");
        return;
    }

    printf("\n---- Command History ----\n");
    command_tstcsprn(command, "\n"] = '\0';

    if (strlen(command) == 0)
    {
        printf("Cannot store an empty command.\n");
    }
    else
    {
        storeCommand(command);
        printf("Command stored successfully.\n");
    }
    break;

case 2:
    displayHistory();
    break;

case 3:
    printf("Enter history number: ");
    scanf("%d", &position);

    retrieveCommand(position);
    break;

case 4:
    validateHistory();
    break;

case 5:
    printf("Exiting program...\n");
    exit(0);

default:
    printf("Invalid choice. Try again.\n");
}

return 0;
}
```

```
root@Shamikahasini:~# nano history.c
root@Shamikahasini:~# gcc history.c -o history
root@Shamikahasini:~# ./history
```

```
===== COMMAND HISTORY =====
```

1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit

Enter your choice: 1

Enter command: ls

Command stored successfully.

```
===== COMMAND HISTORY =====
```

1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit

Enter your choice: 1

Enter command: pwd

Command stored successfully.

```
===== COMMAND HISTORY =====
```

1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit

Enter your choice: 1

Enter command: cd Documents

Command stored successfully.

```
===== COMMAND HISTORY =====
```

1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit

Enter your choice: 1

Enter command: mkdir test

Command stored successfully.

```
===== COMMAND HISTORY =====
```

1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit

Enter your choice: 2

```
----- Command History -----
```

- 1 ls
- 2 pwd
- 3 cd Documents
- 4 mkdir test

```
===== COMMAND HISTORY =====
```

1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit

Enter your choice: |

Observation

The command history program successfully stores commands in a fixed-size history buffer, maintains the most recent commands when the buffer becomes full, retrieves individual commands, displays the complete history, and validates the consistency of the history buffer.

Task-2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, and Test Configurations.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];
    pid_t p1, p2;

    char *cmd1[] = {"ls", NULL};
    char *cmd2[] = {"wc", "-l", NULL};

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    printf("Pipeline created successfully.\n");

    /* Create first process */
    p1 = fork();

    if (p1 < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (p1 == 0)
    {
        /* First command: ls */
        close(pipefd[0]);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    /* Create second process */
    p2 = fork();

    if (p2 < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (p2 == 0)
    {
        /* Second command: wc -l */
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execvp(cmd2[0], cmd2);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    /* Parent process */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Maintain execution order */
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Pipeline execution completed successfully.\n");

    return 0;
}
```

```

root@Shamikahasini:~# nano pipeline.c
root@Shamikahasini:~# gcc pipeline.c -o pipeline
root@Shamikahasini:~# ./pipeline
Pipeline created successfully.
66
Pipeline execution completed successfully.
root@Shamikahasini:~# ls | wc -l
66
root@Shamikahasini:~#

```

Observation:

The pipeline was successfully created using the `pipe()` system call. Two child processes were created using `fork()`, and `dup2()` was used to connect the output of the first command to the input of the second command. The execution order was maintained using `waitpid()`, and the pipeline configuration was tested successfully.

Session-12

Task-1

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, and Test Data Flow.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    printf("Pipe created successfully.\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    {
        /* Child process */

        /* Close unused read end */
        close(pipefd[0]);

        /* Redirect standard output to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        /* Close original write descriptor */
        close(pipefd[1]);
    }
}

```

```

/* Redirect standard output to pipe */
dup2(pipefd[1], STDOUT_FILENO);

/* Close original write descriptor */
close(pipefd[1]);

/* Execute first command */
execlp("ls", "ls", NULL);

perror("execlp");
exit(EXIT_FAILURE);
}
else
{
    /* Parent process */

    /* Close unused write end */
    close(pipefd[1]);

    /* Redirect standard input from pipe */
    dup2(pipefd[0], STDIN_FILENO);

    /* Close original read descriptor */
    close(pipefd[0]);

    /* Wait for child process */
    waitpid(pid, NULL, 0);

    printf("\nData received through pipe:\n");

    /* Execute connected command */
    execlp("wc", "wc", "-l", NULL);

    perror("execlp");
    exit(EXIT_FAILURE);
}

return 0;
}

```

```

root@Shamikahasini:~# nano pipe.c
root@Shamikahasini:~# gcc pipe.c -o pipe
root@Shamikahasini:~# ./pipe
Pipe created successfully.

```

```

Data received through pipe:
68
root@Shamikahasini:~# ls | wc -l
68
root@Shamikahasini:~#

```

Observation

The pipe was successfully created using `pipe()`. The child process executed `ls`, and its standard output was redirected to the pipe. The parent process redirected its standard input from the pipe and executed `wc -l`. Unused file descriptors were closed and process synchronization was performed using `waitpid()`. The data flow between the connected processes was successfully tested.

Task-2

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, and Test Long Pipelines.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define COMMANDS 4

int main()
{
    int pipes[COMMANDS - 1][2];
    pid_t pids[COMMANDS];

    /* Commands for the pipeline */
    char *cmds[COMMANDS][4] = {
        {"ls", NULL},
        {"grep", ".c", NULL},
        {"sort", NULL},
        {"wc", "-l", NULL}
    };

    /* Create multiple pipes */
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            exit(EXIT_FAILURE);
        }
    }

    printf("Multiple pipes created successfully.\n");

    /* Create multiple processes */
    for (int i = 0; i < COMMANDS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
        }

        /* Connect output to next pipe */
        if (i < COMMANDS - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        /* Close all pipe descriptors */
        for (int j = 0; j < COMMANDS - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        /* Execute command */
        execvp(cmds[i][0], cmds[i]);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    /* Parent closes all pipe descriptors */
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    /* Wait for all processes */
    for (int i = 0; i < COMMANDS; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("All processes completed successfully.\n");
    printf("Long pipeline execution completed.\n");

    return 0;
}
```

```
root@Shamikahasini:~# nano multipipe.c
root@Shamikahasini:~# gcc multipipe.c -o multipipe
root@Shamikahasini:~# ./multipipe
Multiple pipes created successfully.
40
All processes completed successfully.
Long pipeline execution completed.
root@Shamikahasini:~# ls | grep ".c" | sort | wc -l
40
root@Shamikahasini:~# |
```

Observation

Multiple pipes were successfully created using a descriptor array. Multiple child processes were launched using `fork()` and connected using `dup2()`. Each process executed its respective command, while the parent process closed all unused descriptors and synchronized process completion using `waitpid()`. The long pipeline was successfully tested and executed.